ILA — Intelligent Learning Assistant

Detailed Build Workflow · Recommendations · Risks · Best Practices

6	24	24	24	24
Build phases	Detailed steps	Recommendations	Risks flagged	Best practices

This is the full ILA build guide — every phase broken into concrete steps, each with a personal recommendation on how to approach it, a flag for what commonly goes wrong, and a best practice to follow. Read one phase at a time. Build, test, confirm it works, then move to the next. The system is fully offline once models are downloaded.

How to read the step cards

■ **Recommendation**
■ **Risk**
■ **Best practice**

Each step card shows the task name, the exact tool or command to use, and three coloured sub-panels: a green recommendation for the best approach, a red risk to watch out for, and a purple best practice to adopt from the start.

1 Environment setup

Get your offline runtime solid before writing a single line of AI code.

■ 1–2 hours

Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh (Linux/Mac) winget install Ollama.Ollama (Windows)
```

Your offline LLM runtime. Handles model downloads, GPU detection, and serves an OpenAI-compatible REST API at localhost:11434 automatically.

■ Recommendation
Install Ollama first — before any Python. Run 'ollama serve' and confirm you see 'Listening on 127.0.0.1:11434' before moving on.

■ Risk
GPU drivers not detected: Ollama silently falls back to CPU. If inference is very slow (< 1 tok/s), your GPU drivers are missing. Run 'ollama info' to verify GPU is listed.

■ Best practice
Always test with 'ollama run phi4-mini' interactively first. If this works, everything else will too.

Create Python virtual environment

```
python -m venv ila_env source ila_env/bin/activate (Linux/Mac) ila_env\Scripts\activate (Windows)
```

Python 3.10 or 3.11 recommended. A venv keeps ILA dependencies isolated from your system Python — critical if you have other projects.

■ Recommendation
Use Python 3.11 specifically. LangChain and ChromaDB have occasional compatibility issues on 3.12 that take time to debug.

■ Risk
Installing packages globally (without a venv) will cause version conflicts later when libraries update. This is one of the most common beginner traps.

■ Best practice
Pin your dependency versions in a requirements.txt immediately: 'pip freeze > requirements.txt'. This makes the project reproducible.

Install core Python packages

```
pip install langchain langchain-community chromadb pip install pymupdf streamlit fastapi uvicorn pip install sentence-transformers plotly pyBKT
```

These 9 packages are the entire stack. LangChain for RAG, ChromaDB for vectors, PyMuPDF for PDFs, Streamlit for UI, FastAPI for the backend, pyBKT for the adaptive model.

■ Recommendation
Install in this order — LangChain first, then ChromaDB. They share dependencies and installing in reverse causes version conflicts.

■ Risk
Installation can take 5–10 minutes due to torch being pulled in by sentence-transformers. If pip hangs, check your network and retry.

■ Best practice
After installing, run 'python -c "import langchain, chromadb, fitz, streamlit"' to confirm all 4 core packages import without errors before proceeding.

Pull LLM and embedding model

```
ollama pull qwen3:8b ollama pull nomic-embed-text
```

qwen3:8b is ~5 GB download. nomic-embed-text is ~274 MB. Both are stored permanently in ~/.ollama/models — you only download once.

■ Recommendation
Pull both models before writing code. You do not want a download interrupting a coding session. On slow connections, do this overnight.

■ Risk
Disk space: each model takes significant space. qwen3:8b = 5 GB, deepseek-r1:14b = 9 GB. Check available disk with 'df -h' first.

■ Best practice
Test immediately: 'ollama run qwen3:8b "Explain Newton's first law in 2 sentences"' — if you get a clean response, your entire LLM stack is working.

2 Document ingestion pipeline

Convert PDFs and notes into searchable vector chunks — the memory of ILA.

■ 1–2 days

PDF text extraction

```
import fitz # PyMuPDF doc = fitz.open('textbook.pdf') text = ''.join([p.get_text() for p in doc])
```

PyMuPDF (imported as 'fitz') is the fastest and most reliable PDF parser. It handles multi-column layouts, preserves paragraph breaks, and is 10x faster than PyPDF2.

Recommendation
Extract text page-by-page, not the whole doc at once. Store page numbers alongside text — you'll need them later to cite sources in answers.

Risk
Scanned PDFs (photos of textbook pages) produce empty text. Always check if len(text.strip()) == 0 after extraction. If so, you need OCR via pytesseract.

Best practice
After extraction, print the first 500 characters and visually inspect. Garbled text, missing spaces, or encoding issues are common and must be fixed before chunking or embeddings will be meaningful.

Text chunking strategy

```
from langchain.text_splitter import RecursiveCharacterTextSplitter splitter = RecursiveCharacterTextSplitter( chunk_size=512, chunk_overlap=64)
```

Chunking splits long text into pieces the LLM can process. RecursiveCharacterTextSplitter tries to split on paragraphs first, then sentences, then words — preserving natural meaning boundaries.

Recommendation
Start with chunk_size=512 and chunk_overlap=64. If answers feel cut off or miss context, increase overlap to 128. Overlap ensures a concept split across two chunks is captured in both.

Risk
Chunk size too large (>1024): retrieval becomes vague — too much irrelevant text. Too small (<200): answers lose context and feel fragmented. 512 is the safe default.

Best practice
Tag every chunk with metadata at creation time: {source_file, page_number, chapter, topic}. This metadata is stored free in ChromaDB and is invaluable for showing students which page an answer came from.

Generate embeddings

```
import ollama response = ollama.embeddings( model='nomic-embed-text', prompt=chunk_text ) vector = response['embedding']
```

Each chunk is converted to a dense vector of ~768 numbers representing its semantic meaning. Similar text produces similar vectors — this is what enables semantic search rather than keyword matching.

Recommendation
Always use the same embedding model for indexing AND querying. Mixing models (e.g. indexing with nomic-embed-text, querying with bge-m3) gives completely wrong retrieval results — a silent, hard-to-debug failure.

Risk
Embedding 1000 chunks takes ~2–5 minutes on CPU. If you have many PDFs, batch your embed calls (10–20 at a time) and add a progress bar so you know the pipeline hasn't frozen.

Best practice
Cache your embeddings. Once a PDF is indexed, store a hash of the file. On re-upload, skip re-indexing if the hash matches. This saves minutes of wait time as the material library grows.

Store vectors in ChromaDB

```
import chromadb client = chromadb.PersistentClient(path='./ila_db') col = client.get_or_create_collection('study_material') col.add(documents=chunks, embeddings=vecs, ids=ids, metadatas=meta)
```

ChromaDB stores your vectors persistently to disk at './ila_db'. get_or_create_collection is safe to call on startup — it creates the collection the first time and loads it on subsequent runs.

Recommendation
Use PersistentClient (not InMemoryClient) from the very first line of code. Switching later means re-indexing all your documents.

Risk
Duplicate IDs will silently overwrite existing entries in ChromaDB. Generate IDs as '{filename}_{page}_{chunk_index}' to guarantee uniqueness across multiple uploaded PDFs.

Best practice
After indexing, run a test query: col.query(query_texts=['test'], n_results=3) and print the returned documents. If they look semantically relevant, your entire ingestion pipeline is working correctly.

3 RAG query engine

Student asks a question → retrieve relevant context → LLM answers from your material.

■ 1 day

Embed the student query

```
q_vec = ollama.embeddings(
model='nomic-embed-text',
prompt=student_question )['embedding']
```

The student's question is converted to a vector using the exact same embedding model used during indexing. This puts the question and your documents into the same vector space for comparison.

■ Recommendation
Lowercase and strip the query before embedding. 'What is Newton?' and 'what is newton?' produce slightly different vectors — normalizing improves retrieval consistency.

■ Risk
Never use a different embedding model for queries than for indexing. This is the single most common RAG bug and produces retrieval results that look random.

■ Best practice
Log every query vector at first. If retrieval results look wrong, you can replay the exact query vector against ChromaDB in isolation to debug whether the problem is in embedding or retrieval.

Retrieve top-k context chunks

```
results = collection.query(
query_embeddings=[q_vec], n_results=4,
include=['documents','metadatas','distances']
)
```

ChromaDB finds the 4 most semantically similar chunks to your query. The 'distances' field tells you how confident the match is — lower distance = more relevant.

■ Recommendation
Start with n_results=4. Too few (1–2) and the LLM lacks context. Too many (8+) and you overflow the LLM's context window and slow down inference significantly.

■ Risk
If distance scores are all > 1.5, no relevant content was found — the student is asking about something not in their uploaded material. Handle this gracefully: tell the student rather than hallucinating an answer.

■ Best practice
Always include metadata in your query results. Showing the student 'from Chapter 3, page 47' next to an answer builds trust and helps them find the source — a key feature for a study tool.

Build the RAG prompt

```
system = ''' You are ILA, an AI tutor. Answer
using ONLY the context below. If the answer
isn't in the context, say so clearly. '''
prompt = system + context_text +
'\n\nQuestion: ' + question
```

The prompt structure is: (1) system role, (2) retrieved context passages, (3) student question. The LLM is instructed to answer only from the provided context — preventing hallucination of facts not in their material.

■ Recommendation
Explicitly instruct the LLM to say 'I don't know' if the answer isn't in the context. Without this, it will confidently make something up. This is the most important prompt instruction for a tutoring use case.

■ Risk
Context window overflow: if you pass too many chunks, the LLM truncates your material silently. With qwen3:8b, keep total prompt under ~3000 tokens (about 4 chunks of 512 tokens each).

■ Best practice
Version your prompt templates in a separate prompts.py file. Tweaking prompts is the primary way to improve ILA's quality — you'll iterate on these constantly.

Generate and stream the answer

```
import requests response = requests.post(
'http://localhost:11434/api/generate',
json={'model':'qwen3:8b', 'prompt':prompt,
'stream':True} )
```

POST to Ollama's local API with stream:True. This sends tokens as they're generated, making the UI feel responsive rather than freezing for 10 seconds. Process the NDJSON stream line by line.

■ Recommendation
Always use streaming. A non-streaming call blocks your entire app for 5–15 seconds on a typical explanation response. Students will think the app has crashed.

■ Risk
Ollama can only run one generation at a time per model. If two students use ILA simultaneously (in a shared setup), the second request queues. For single-student use this is fine.

■ Best practice
Log every (question, retrieved_chunks, answer) triple to a file during development. This is your primary debugging tool — when ILA gives a bad answer, this log tells you exactly what context it had access to.

4

Quiz generation + performance tracking

Auto-generate quizzes from material. Track mastery per topic. Feed data to the adaptive engine.

2–3 days

Quiz generation via LLM

```
quiz_prompt = f''' Generate 4 multiple-choice questions from this text. Output ONLY valid JSON: [{{'question':..., 'options':[...], 'answer':..., 'topic':...}}] Text: {context} '''
```

Prompt the LLM to produce structured quiz questions from retrieved chunks. Asking for JSON output makes parsing reliable. The 'topic' field per question is used to tag performance data.

■ Recommendation
Ask for JSON output and validate it with json.loads() before using. Add 'Output ONLY valid JSON, no explanation text' to the prompt — without this, the LLM often wraps JSON in markdown code blocks.

■ Risk
LLMs occasionally generate JSON with syntax errors (trailing commas, missing quotes). Wrap json.loads() in a try/except and re-prompt once on failure before showing an error to the student.

■ Best practice
Generate quizzes from the same chunks used to answer the student's last question. This creates a natural study → test → review cycle grounded in what was just discussed.

Store quiz responses in SQLite

```
import sqlite3 conn = sqlite3.connect('ila_students.db') cursor.execute(''' CREATE TABLE IF NOT EXISTS quiz_results (id, student_id, topic, correct, timestamp, question) ''')
```

Every quiz attempt is stored locally in SQLite. This is the raw data that powers the adaptive engine. SQLite needs zero configuration and comes built into Python.

■ Recommendation
Always store the raw question text alongside correct/incorrect. Storing only a boolean discards information you'll need later to understand patterns in what the student gets wrong.

■ Risk
No schema versioning from the start leads to painful migrations later. Add a 'schema_version' table immediately. When you add columns, increment it and write a migration script.

■ Best practice
Index your quiz_results table on (student_id, topic) immediately: 'CREATE INDEX idx_student_topic ON quiz_results(student_id, topic)'. Without this, mastery queries get slow once a student has 500+ records.

Compute topic mastery with BKT

```
from pyBKT.models import Model model = Model(seed=42) model.fit(data=student_data) mastery = model.predict(data=student_data)
```

Bayesian Knowledge Tracing (BKT) estimates the probability that a student has mastered a topic based on their sequence of correct/incorrect answers. It's the same algorithm used by Khan Academy.

■ Recommendation
Fit a separate BKT model per topic — not one global model. Topics like 'calculus derivatives' and 'Newton's laws' have very different learning curves and shouldn't share parameters.

■ Risk
BKT needs at least 5–8 responses per topic before its estimates become reliable. Below that threshold, use a simple percentage correct as a fallback mastery score.

■ Best practice
Recompute mastery scores after every quiz session, not in real-time during a session. Batch recomputation is faster and the small delay (< 1 second) is imperceptible to students.

Adaptive topic selection

```
MASTERY_THRESHOLD = 0.75 weak = [t for t, score in mastery.items() if score < MASTERY_THRESHOLD] next_topic = sorted(weak, key=lambda t: mastery[t])[0]
```

After each quiz session, identify topics below 75% mastery. Sort weakest first. The next quiz session focuses on these. This is the core adaptive loop — what makes ILA intelligent.

■ Recommendation
Set the mastery threshold at 0.75 initially — not 1.0. Requiring 100% mastery on every topic creates frustration. 75% is the research-backed threshold for moving forward.

■ Risk
Purely focusing on weak topics can demotivate students. Mix 70% weak-topic questions with 30% strong-topic review. This prevents forgetting previously mastered material.

■ Best practice
Always tell the student why they're seeing certain questions: 'Based on your recent performance, here are questions on Newton's Laws.' Transparency in the adaptive logic builds trust in the system.

5 Adaptive explanation engine

Personalise how the LLM explains based on the student's mastery level and history.

■ 2 days

Build the student profile store

```
profile = { 'student_id': 'stu_001', 'level': 'beginner', # beginner/intermediate/advanced 'weak_topics': ['integration', 'optics'], 'explanation_style': 'analogies', 'session_count': 12 }
```

The student profile is the state that makes ILA personalised. It stores their current understanding level, which topics they struggle with, and their preferred explanation style.

■ Recommendation
Infer the level automatically from mastery scores rather than asking. average mastery < 0.5 → beginner, 0.5–0.8 → intermediate, > 0.8 → advanced. Students find self-reported level selection stressful and inaccurate.

■ Risk
Profiles stored only in memory are lost on restart. Persist to SQLite immediately — a crashed session should not reset a student's progress.

■ Best practice
Build a profile reset button in the UI. Students starting a new subject should be able to clear their history cleanly without deleting the whole database.

Dynamic system prompt injection

```
system = f''' You are ILA, a patient AI tutor. Student level: {profile['level']}. Struggling with: {', '.join(profile['weak_topics'])}. Use {profile['explanation_style']} when explaining. For beginner: use simple language and everyday analogies. '''
```

The student profile is injected directly into the LLM's system prompt before every response. This is how the same LLM produces different explanations for different students with zero retraining.

■ Recommendation
Keep the injected profile section under 200 tokens. Longer profiles eat into context space that should be used for retrieved document chunks.

■ Risk
Prompt injection is a real security risk if student-entered text gets into the system prompt. Never interpolate raw user input directly. Only inject values from your own profile store.

■ Best practice
A/B test your system prompt variations. Keep a log of which prompt version was used for each session — this lets you compare explanation quality when you iterate on the prompt.

Confusion detection and re-explanation

```
def is_confused(topic, history): recent = [r for r in history if r['topic']==topic][-5:] return len(recent) >= 2 and sum(r['correct'] for r in recent) / len(recent) < 0.5
```

If a student answers fewer than 50% of recent questions on a topic correctly, or asks about the same concept twice, trigger a re-explanation with a simpler prompt: 'Explain this as if to a complete beginner using a real-world example.'

■ Recommendation
Don't immediately simplify on the first wrong answer — that feels patronising. Trigger simplified re-explanation only after 2+ consecutive failures or an explicit 'I don't understand' from the student.

■ Risk
Over-simplifying is a real risk. A student who has mastered 80% of a topic but fails one edge-case question should not receive a kindergarten explanation. Check overall mastery before simplifying.

■ Best practice
Log every triggered re-explanation with the before/after mastery scores. This is your ground truth for whether the simplification actually helped — invaluable data for improving ILA over time.

Spaced repetition scheduler

```
# SM-2 algorithm def next_review(ease, interval, quality): if quality >= 3: interval = interval * ease ease = ease + 0.1 - (5-quality)*(0.08+(5-quality)*0.02) else: interval = 1 # reset return max(1, interval), max(1.3, ease)
```

Spaced repetition schedules when topics should be reviewed based on the forgetting curve. Topics answered correctly get pushed further into the future. Missed topics are surfaced the next day. Uses the SM-2 algorithm.

■ Recommendation
Implement SM-2 before adding anything fancier. It's 10 lines of code, backed by decades of research, and is more effective than most ML-based alternatives for simple recall tasks.

■ Risk
Students find daily required sessions demotivating. Make spaced repetition a suggestion ('3 topics due for review today') rather than a blocker that prevents accessing new material.

■ Best practice
Store the ease factor and interval per (student, topic) pair in SQLite. The ease factor is the key state — losing it means the schedule restarts from scratch.

6 Frontend + API layer

Wrap everything in a usable interface. Streamlit gives you a full app in ~150 lines of Python.

■ 2–3 days

FastAPI backend

```
from fastapi import FastAPI app = FastAPI() @app.post('/ask') async def ask(question: str, student_id: str): # RAG pipeline here return {'answer': answer, 'sources': sources}
```

FastAPI wraps your RAG pipeline in a clean HTTP API. Endpoints: POST /upload (ingest PDF), POST /ask (RAG query), GET /quiz (generate questions), GET /progress (mastery scores).

Recommendation
Build the FastAPI backend even if you're only building for yourself. It separates your AI logic from the UI cleanly, and makes ILA trivially easy to extend later (mobile app, browser extension, etc.).

Risk
Don't run CPU-heavy operations (embedding, LLM calls) synchronously in FastAPI. Use BackgroundTasks or asyncio for long operations — otherwise the API hangs while waiting for Ollama.

Best practice
Enable FastAPI's auto-generated docs: visit localhost:8000/docs while the server is running. This gives you a free interactive API tester that's invaluable during development.

Streamlit chat interface

```
import streamlit as st st.title('ILA — Your Study Assistant') if prompt := st.chat_input('Ask anything from your material...'): with st.chat_message('assistant'): st.write_stream(get_ila_response(prompt))
```

Streamlit's chat components (st.chat_message, st.chat_input, st.write_stream) give you a production-quality chat UI in 15 lines. write_stream renders streaming tokens in real time.

Recommendation
Store chat history in st.session_state['messages'] so it persists during a session. Without this, the chat history disappears on every Streamlit re-run (which happens on every interaction).

Risk
Streamlit re-runs the entire script on every user action. Heavy operations (loading ChromaDB, initialising the Ollama client) must be wrapped in @st.cache_resource or they re-execute every message — making ILA unusably slow.

Best practice
Add a source expander below every answer: with st.expander('Sources'): show the retrieved chunk texts and page numbers. This is the single most trusted feature in study tools.

PDF upload and indexing UI

```
uploaded = st.file_uploader( 'Upload your study material', type=['pdf'], accept_multiple_files=True ) if uploaded: with st.spinner('Indexing...'): ingest_pdf(uploaded)
```

The file uploader triggers the full ingestion pipeline: extract → chunk → embed → store. A spinner keeps the student informed. Show a success message with the chunk count when done.

Recommendation
Show a progress bar during indexing, not just a spinner. 'Indexed 47 of 312 chunks...' is far less anxious than a spinning circle with no indication of progress.

Risk
Large PDFs (300+ pages) can take 5–10 minutes to index. If the student closes the browser during indexing, the process should continue in the background — use FastAPI background tasks rather than running indexing in the Streamlit thread.

Best practice
Store indexed file names in SQLite with their hash. On re-upload of the same PDF, skip re-indexing and show 'Already indexed'. Students often accidentally upload the same file twice.

Progress dashboard

```
import plotly.express as px mastery_df = get_mastery_dataframe(student_id) fig = px.bar(mastery_df, x='topic', y='mastery', color='mastery', color_continuous_scale='RdYlGn') st.plotly_chart(fig)
```

The progress dashboard shows mastery per topic as a colour-coded bar chart (red = weak, green = strong), a quiz streak counter, a review schedule ('3 topics due today'), and total study time.

Recommendation
Make the mastery chart the first thing students see when they open ILA, not the chat. Students are more motivated when they can see their progress clearly before starting a session.

Risk
Avoid vanity metrics (total questions answered, total time). Focus on mastery scores and weak-topic lists — these are actionable. Vanity metrics can mask lack of real progress.

Best practice
Add a 'What should I study today?' button that reads the mastery scores and returns a prioritised list. This removes decision fatigue and is one of the most valued features in adaptive learning systems.

Complete recommended tool stack

Library	Role in ILA	Install	License
Ollama	LLM + embedding runtime	`ollama.com`	Free, Apache 2.0
qwen3:8b	Primary tutoring LLM	`ollama pull qwen3:8b`	Free, Apache 2.0
nomic-embed-text	Embedding model	`ollama pull nomic-embed-text`	Free, Apache 2.0
ChromaDB	Vector store	`pip install chromadb`	Free, Apache 2.0
LangChain	RAG orchestration	`pip install langchain`	Free, MIT
PyMuPDF (fitz)	PDF extraction	`pip install pymupdf`	Free, AGPL
FastAPI	Backend REST API	`pip install fastapi uvicorn`	Free, MIT
Streamlit	Frontend UI	`pip install streamlit`	Free, Apache 2.0
SQLite	Student data store	`built into Python`	Free, Public domain
pyBKT	Knowledge tracing	`pip install pyBKT`	Free, MIT
Plotly	Dashboard charts	`pip install plotly`	Free, MIT
pytesseract	OCR for scanned PDFs	`pip install pytesseract`	Free, Apache 2.0
sentence-transformers	Alt embedding pipeline	`pip install sentence-transformers`	Free, Apache 2.0

Build checklist

Ph.1	■ Install Ollama → run 'ollama run qwen3:8b' → confirm it responds
Ph.1	■ Create venv → install all packages → confirm imports work
Ph.2	■ Write ingest.py: PDF → chunks → embed → ChromaDB. Test on ONE real PDF.
Ph.2	■ Query ChromaDB manually and inspect returned chunks before building the LLM layer.
Ph.3	■ Write query.py: question → embed → retrieve → prompt → LLM answer. Test in terminal.
Ph.3	■ Verify answers are grounded in doc chunks, not hallucinated. Check source metadata.
Ph.4	■ Write quiz_gen.py: prompt LLM for JSON quiz. Validate JSON output. Store in SQLite.
Ph.4	■ Implement BKT per topic. Test with synthetic correct/incorrect sequences.
Ph.5	■ Build student_profile.py: load/save/update profile in SQLite.
Ph.5	■ Inject profile into system prompt. Verify qwen3:8b adjusts explanation depth.
Ph.6	■ Build Streamlit app.py: chat + file uploader. Confirm streaming works end-to-end.
Ph.6	■ Add progress dashboard with mastery bar chart. Wire to SQLite quiz data.
Ph.6	■ Add FastAPI backend. Confirm all endpoints work via /docs interface.
Ph.—	■ Full end-to-end test: upload a PDF → ask 5 questions → take a quiz → check dashboard.